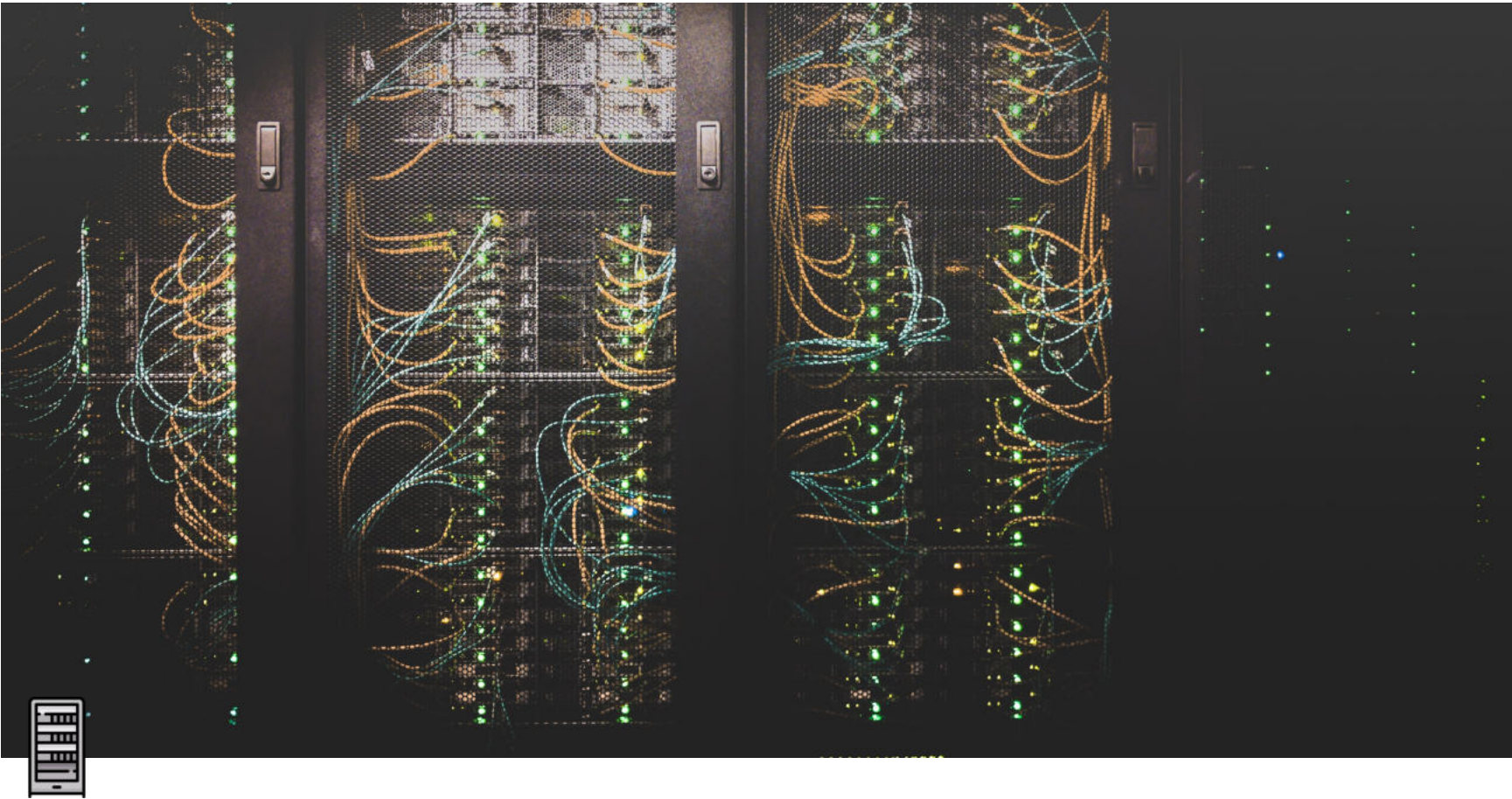




Combining commands and files

Type	Lecture
Date	@January 9, 2022
Lecture #	1
Lecture URL	https://youtu.be/Lcx9UsS7y8Y
Notion URL	https://21f1003586.notion.site/Combining-commands-and-files-2476c1091a704743840ae8b76ab078c9
Week #	3

Executing multiple commands

- `command1; command2; command3`
 - Each command will be executed one after the other
- `command1 && command2 && command3`
 - This works as a logical AND
 - The subsequent commands after `command-n` will not run if the previous command resulted in an error
- `command1 || command2 || command3`
 - This works as a logical OR
 - The subsequent commands after `command-n` will not run if the previous command resulted in a success

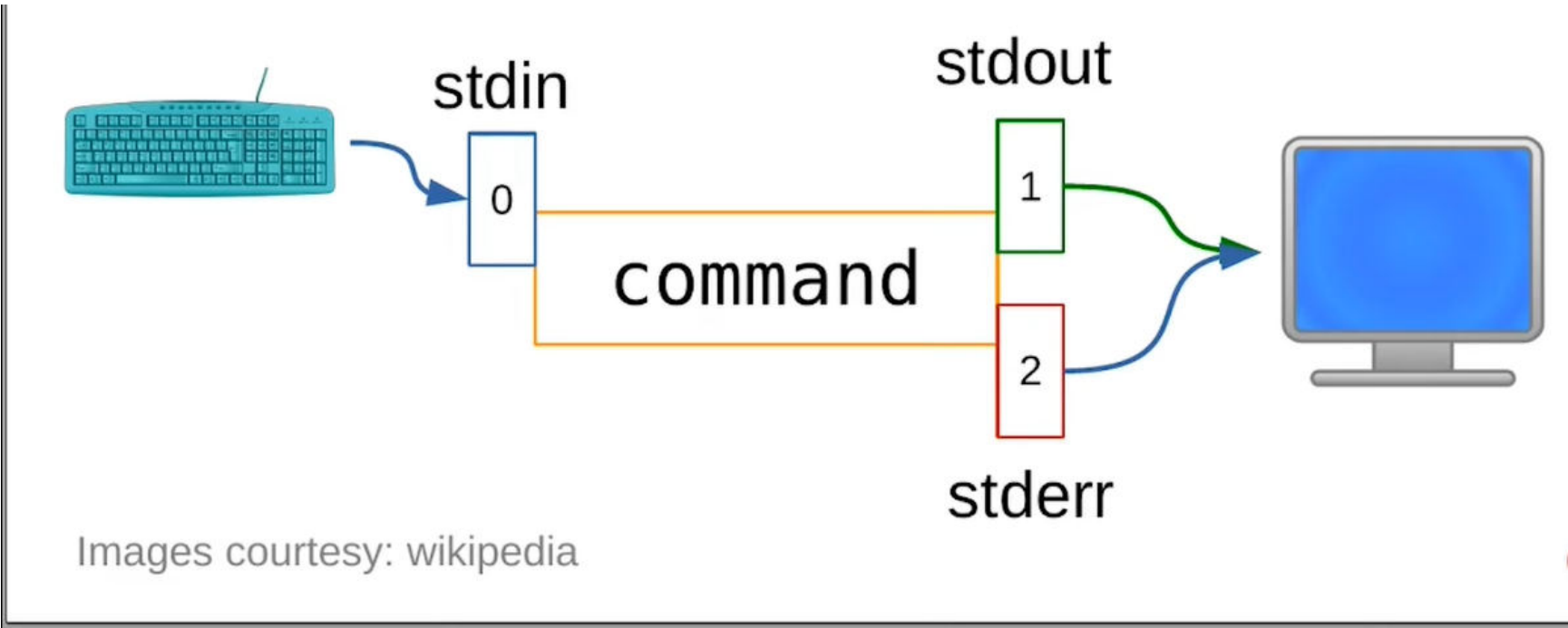
`(<command>)`

We can run any command enclosed within parentheses to execute them in a subshell, and returned back the result

We can execute a subshell within a subshell too

```
kashif@Zen:~$ echo $BASH_SUBSHELL
0
kashif@Zen:~$ (echo $BASH_SUBSHELL)
1
kashif@Zen:~$ (echo $BASH_SUBSHELL; (echo $BASH_SUBSHELL))
1
2
```

File descriptors

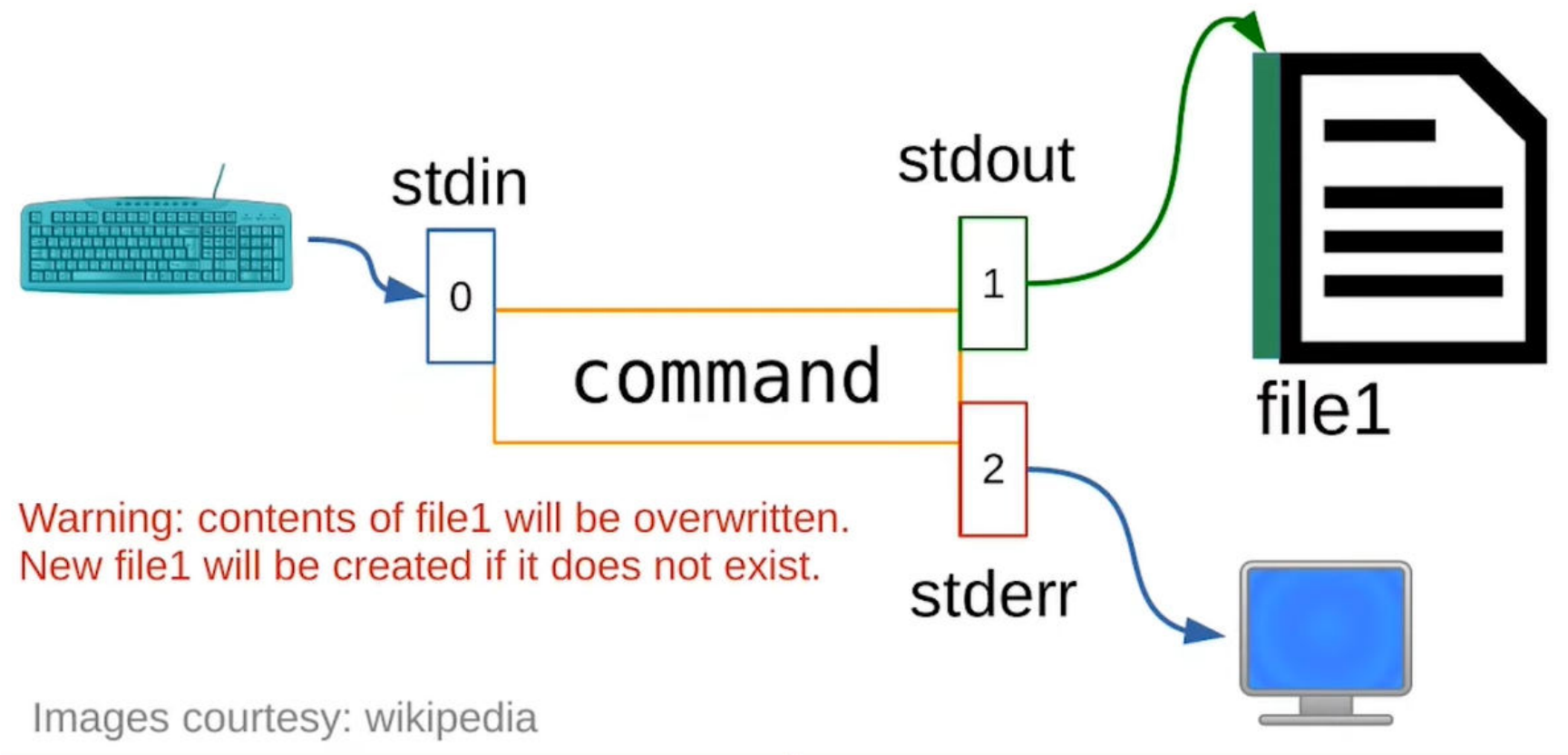


Every command in Linux has 3 file descriptors

- `stdin` (0)
 - It is a pointer to a stream that is coming from the keyboard (or the user input)
- `stdout` (1)
 - Points to the screen where the output is made
- `stderr` (2)
 - Points to the screen where the output is made

`command > file1`

- The output of the `command` should be written to `file1`



Create a file using `cat` command

`cat > filename`

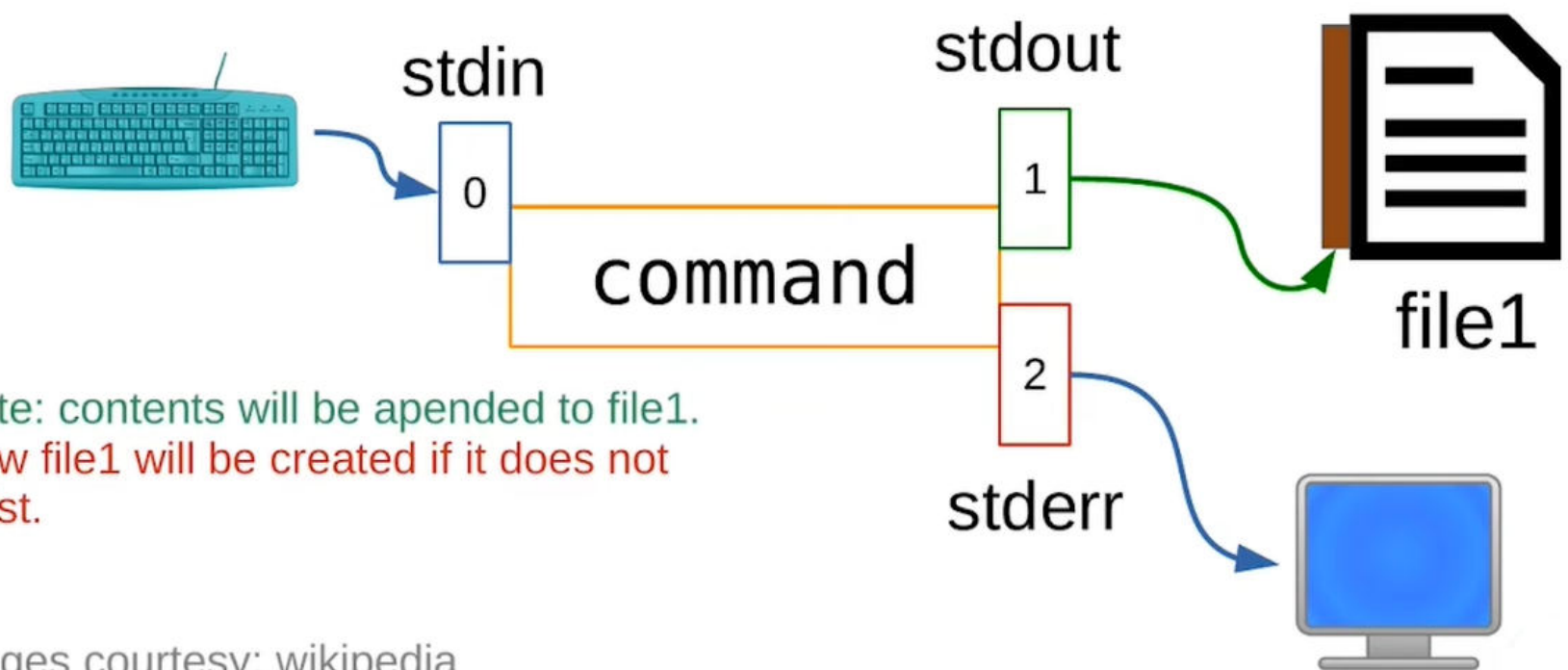
When we type this command, the `cat` command is supposed to receive the input from a file that is listed in the command line, but instead, we left that intentionally blank

So, the `cat` command, instead, reads the content from the `stdin`, i.e. the keyboard

To exit, press `Ctrl + D`

`command >> file1`

- The output of `command` will be appended to `file1`



Similarly, we can use `>>` instead of `>` while creating a new file using the `cat` command